Session 2: Research Pipelines

From Data to Results

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code can do. How it works: models, context, and the AI skill gap. CLAUDE.md.

Session 2: How to use it on a real research task — and stay in control.

Today you will:

1. Merge two datasets with explicit instructions
2. Estimate a model and verify the output
3. Generate robustness checks
4. Produce a LaTeX results table

The methods are straightforward. The skill is the workflow.

Start-of-Session Ritual

Work on a copy, not inside the shared repo. Do this now:

Mac / Linux

```
# Already cloned? Update:
cd ~/claude-code-workshop && git pull

# First time:
git clone https://github.com/
  bradyallardice/claude-code-workshop.git

# Copy and open
cp -r session_2/ ~/my_workspace/session_2/
cd ~/my_workspace/session_2
code .
```

Windows (PowerShell)

```
# Already cloned? Update:
cd ~\claude-code-workshop; git pull

# First time:
git clone https://github.com/
  bradyallardice/claude-code-workshop.git

# Copy and open
Copy-Item -Recurse session_2 `
  ~\my_workspace\session_2
cd ~\my_workspace\session_2
code .
```

If `git pull` Fights Back

If you committed work inside the repo, `git pull` will complain. Run this recovery in the course repo, then continue with Steps 2–3:

```
git stash -u # save uncommitted edits
git branch backup-prework # save any local commits
git fetch origin
git reset --hard origin/student # match the server
git stash pop # skip if stash was empty
```

Your old commits live on `backup-prework`. If that name exists, add a suffix: `backup-prework-2`.

Strategies for Working Within the Context Window

Recall from Session 1: Claude Code can only "see" a limited amount of text at once. Errors compound when it loses track of earlier decisions.

Five strategies to work within this constraint:

1. Keep tasks focused
2. Maintain project documentation
3. Summarize work between conversations
4. Validate iteratively
5. Write token-aware prompts

What Happens When the Context Window Fills Up

As a conversation grows, Claude Code eventually runs out of room.
When this happens, it **auto-compacts**: it summarizes older parts of the conversation to free up space.

What auto-compacting does:

- Replaces earlier messages with a compressed summary
- Keeps recent messages and tool results intact
- Happens automatically. You will see a notice in the chat

The risk: Details from early in the conversation (variable definitions, decisions you discussed, constraints you mentioned) may be lost or simplified in the summary.

You can also compact manually:
Type `/compact` to force a summary and free up context space.

This is why CLAUDE.md matters: it survives compacting because it is reloaded from disk, not from conversation history.

Strategy 1: Keep Tasks Focused

One topic, one conversation.

Avoid:

- "Clean the data, build the panel, run the regressions, and make tables"
- Long conversations that drift across topics
- Asking Claude Code to hold your entire project in its head

Instead:

- "Clean and reshape the raw census file"
- Start a new conversation for each distinct task
- Let each conversation do one thing well

Why

A focused task fits comfortably in the context window. A sprawling task forces Claude Code to forget earlier steps, and that is where silent errors enter.

Demo: Keeping Tasks Focused

Live demo — follow along on your own machine

Open the Session 2 project in VS Code and start Claude Code from the project folder.

Conversation 1: one narrow task

```
Read docs/codebook.md and list the variables in
data/swiss_franc_survey.csv with a one-line description
of each. Do not modify any files.
```

Conversation 2: start a new conversation for the next task

```
Read data/swiss_franc_survey.csv and report how many
respondents are in each treatment group. Do not modify
any files.
```

Strategy 2: Maintain Project Documentation

Claude Code reads certain files **automatically** every time it starts.

Keep these up to date:

- `CLAUDE.md`: conventions, constraints, variable definitions
- `README.md`: project structure, what each directory contains
- Directory-level `CLAUDE.md` files for subfolders with specific rules

The Payoff

Good documentation means Claude Code starts every conversation already knowing your conventions, without you spending tokens re-explaining them.

Demo: What a Well-Organized Project Looks Like

Live demo

A real research project directory, shown as an example of good project documentation.

What to look for:

- How the folders are organized
- What `CLAUDE.md` and `README.md` say, and what they leave out

Strategy 3: Summarize Work Between Conversations

When you finish a conversation, the context is gone. The next conversation starts fresh.

Before ending a conversation:

1. Ask Claude Code to summarize what it did and what remains
2. Save that summary (in a file, a commit message, or your notes)
3. Paste the summary into the *next* conversation as context

Example prompt:

```
Summarize what we did in this session: what files
were changed, what decisions were made, and what
still needs to be done.
```

Think of it like a handoff memo between research assistants.
The next "assistant" knows nothing unless you brief it.

Demo: Summarizing a Session

Live demo — follow along on your own machine

Ask Claude Code for a handoff summary at the end of a conversation.

What we will do:

1. Ask for a summary of what was done and what remains
2. Save it to a file
3. Use it to start the next conversation

Strategy 4: Validate Iteratively

The Compounding Error Problem

If step 1 is wrong and you do not catch it, steps 2–5 build on that mistake. By the time you check the final output, the error is buried.

Check each step before moving to the next:

1. Clean the data → *verify row counts and distributions*
2. Construct variables → *spot-check values against known cases*
3. Run the merge → *confirm pre- and post-merge row counts*

Slower per step, but **much faster overall** because you never have to unwind a chain of errors you didn't catch.

We practice this in the validation section later today.

Strategy 5: Token-Aware Prompting

Every word in your prompt costs tokens. Make them count.

A good prompt tells Claude Code three things:

1. **What files matter:** "Read `data/raw/census.csv` and `scripts/clean.py`"
2. **What the goal is:** "Create a county-year panel with one row per county per decade"
3. **What constraints apply:** "Keep all 3,143 counties; do not drop rows with missing population"

Vague:

"Clean up the data and make it ready for analysis"

Specific:

"Read `census.csv`, drop rows where `year` < 1950, and save as `panel_clean.csv`"

The Rule

Be specific about *inputs*, *outputs*, and *constraints*. Be brief about everything else. Context is finite; spend it on what matters.

Demo: A Good Prompt

Live demo — follow along on your own machine

A prompt that names the files, the goal, and the constraints.

What to look for:

- Which files does the prompt point Claude Code to?
- What is the goal, stated in one sentence?
- What constraints are spelled out?

The Core Workflow

	Step	Who	If something is wrong
1.	Describe	You	Rethink before sending
2.	Review plan	You *	Reject, re-instruct
3.	Generate & validate	Claude Code + You	Read the code, run the tests, re-instruct
4.	Save or re-instruct	You	Keep the result (git in Session 3)

* = checkpoint where you can catch a problem before anything gets saved.

Validation is not a separate step — it happens **continuously** throughout steps 1–3.

The pattern

When something goes wrong, go back to **whichever step actually fixes the problem**: a bad plan means rewriting the instruction (step 1), not just re-running the same one. Diagnose first, then go back to the right step.

Step 1: Describe — Write It Down First

Before you type anything into Claude Code, write down your instruction.
If you cannot state the task precisely on paper, Claude Code cannot execute it precisely in code.

A good instruction specifies three things:

1. **Inputs:** "Read `data/survey.csv` and `data/demographics.csv`"
2. **Goal:** "Left-join demographics onto the survey by respondent ID"
3. **Constraints:** "Output must have exactly 2,044 rows. Do not drop respondents."

Vague:
"Merge the datasets and run a regression"

Specific:
"Left-join demographics onto the survey by respondent_id. Use control as the reference. Run a weighted logistic regression."

Be specific about inputs, outputs, and constraints. Be brief about everything else.

The Three Execution Modes

The most important configuration decision you will make.

Plan Mode: Reviews approach with you *before* acting

Default Mode: Acts, but asks permission for "significant" steps

Auto-Accept: Executes everything without asking

↓ More autonomy, less oversight ↓

How it works:

- Claude Code analyzes your request
- Proposes a detailed plan (files to read, changes to make, approach)
- Waits for your explicit approval before acting
- You review, approve, modify, or reject

When to use:
Any task where the *approach* matters, not just the output.
Data transformation, variable construction, merges, anything analytical.

Why It Matters

The plan is your checkpoint. This is where you catch "I am going to do an inner join" *before* it happens, not after 300 observations are gone.

How it works:

- Claude Code executes steps autonomously
- Pauses for actions *it* considers significant
- Uses its own judgment about when to ask

When to use:
Routine tasks in established projects where you trust the general approach.

The Risk for Academics

Claude Code's judgment about what is "significant" may not match yours.
It might pause before writing a file but **not** before silently recoding a variable.

How it works:

- Executes everything without asking
- Reads, writes, runs, and modifies freely
- Stops only on errors

When to use:

- File reformatting
- Boilerplate generation
- Documentation

When not to use:
Anything involving your data or analysis.
The speed gain is not worth the loss of checkpoints.

The Key Insight on Modes

Different Relationships with the Agent

Plan mode = you **direct** the work
Auto-accept = you **review** the work after the fact

For published research, directing is almost always safer than reviewing after the fact.

Why: Errors you *prevent* at the planning stage are cheaper than errors you *catch* in the output, or worse, errors you don't catch at all.

Live Demo: Plan Mode vs. Auto-Accept

Same task, two modes.

Watch for:

- What does a plan look like?
- How do you evaluate it?
- What does the experience feel like in each mode?

Step 2: Review the Plan

In plan mode, Claude Code proposes an approach before acting.

What to look for:

- Does it read the right files?
- Does the merge/analysis strategy match what you specified?
- Does it handle edge cases (missing data, extra rows, reference categories)?
- Does it make any *analytical decisions* you did not ask for?

The key question

Is there anything in this plan that I did not explicitly ask for?
If yes: reject or modify before it runs.

You do not always need plan mode. For simple tasks — summary statistics, reformatting, boilerplate — steps 1–3 collapse into one. But when uncertain, *use it*.

Step 3: Generate and Validate

Claude Code writes and runs the code. Your job is not done.

Read what Claude Code produced:

- Does the script do what the plan said it would?
- Are variable names and file paths correct?
- Did it make any choices the plan did not mention?

Common surprises at this stage:

- Silently dropping rows with missing values
- Choosing a default reference category you did not specify
- Using unweighted estimation when you asked for weighted
- Adding "helpful" data cleaning steps you did not request

The habit

Read the code, not just the output. Output can look correct even when the code is wrong.

Verification: Code Can Run and Still Be Wrong

Claude Code can write code that runs, prints a plausible-looking table, and is still **scientifically wrong**.

None of these raises an error:

- A merge that duplicates rows when keys repeat
- A regression that silently drops every row with a missing control
- Standard errors clustered at the wrong level
- A reference category chosen alphabetically instead of by design

Two different questions

Software tests ask: does the code run and behave as intended?
Research tests ask: are the numbers right?

Read the Code

Claude Code is not deterministic — the same prompt can produce different code each time. But **code itself is deterministic.** If you read the code and understand what it does, you can be confident in the results, the same way you would be with code you wrote yourself.

This is the most fundamental verification:

- Read the script Claude Code produced, line by line
- Confirm it does what you asked — not more, not less

The principle

You are not trusting Claude Code. You are trusting *the code*. Reading does not scale to every number, which is why we also write tests.

Layer 1: Software Tests Check That the Code Works

Test	It asks	Example from today's data
Smoke	Does the project run at all?	Packages import; both CSV files load
Unit	Does one function behave correctly?	The recode returns 1 for "some" or "big" and 0 for "none"
Integration	Do the pieces fit together?	The cleaned data has every column the model script uses
End-to-end	Does the whole pipeline run?	Raw data to final table from a single command
Regression (software sense)	Did a change alter verified results?	After refactoring, N and coefficients are unchanged
Edge-case and negative	Do unusual or bad inputs fail loudly?	A duplicate ID stops the merge instead of multiplying rows

Layer 2: Research Tests Check That the Results Are Right

Test	It asks	Example from today's data
Data integrity	Are keys, types, and ranges valid?	`respondent_id` is unique; age is between 18 and 85
Merge integrity	Did the join keep and match what it should?	2,044 rows in and out; unmatched respondents are counted
Sample accounting	Where did every row go?	N for each model, with a reason for every drop
Variable construction	Does each variable match its definition?	`supports_intervention` is 1 exactly when the answer is "some" or "big"
Estimation specification	Is the model the one you intended?	Control is the reference arm; survey weights are used
Independent numerical check	Does a separate calculation agree?	A weighted share recomputed straight from the raw CSV

Independent Tests

A test only helps if it can fail for a reason the code cannot also fail for.

Independent:
Expected values come from outside the code under test: computed by hand from the raw file, by a different method, or from a published result.

Not independent:
Running the pipeline and pasting its output in as the "expected" value. The test can only confirm that the code agrees with itself.

Example prompt:

```
Write and run tests for these values, which I computed
from the raw file:
- Respondent 9 has treatment = "cntrl", fx_status = "none"
- The control group has exactly 480 respondents
- The weighted share supporting intervention, among
  non-missing responses, is approximately 0.486
Do not take expected values from the analysis script.
```

Write Down Expectations First, Then Assert Them

The Pre-Registration Analogy

Just as you pre-register hypotheses before seeing results, write down what a correct transformation should produce *before* you run it.

Before a merge or transformation, state:

- Expected rows $\times$ columns
- Which columns, and their types
- How missing values are handled
- A row you can trace by hand

Then assert what must be true:

- A left join keeps every row of the left table
- A new control changes N only through missing values
- A share lies between 0 and 1
- Treatment arms sum to the total sample

A mismatch is either a bug or a misunderstanding. Both are worth catching early.

Trace Samples End to End

Aggregate checks can pass while individual records are wrong.
Follow specific records through every stage of the pipeline.

Raw file → Merged → Recoded → Estimation sample → Table

- Choose 3–5 respondents *on purpose*: typical, missing value, no match in file 2
- Print each one's values at every stage, side by side
- Tie every number in the final table (N, coefficients) to the rows behind it

Example prompt:

```
Pick one typical respondent, one with a missing outcome, and
one absent from respondent_demographics.csv. Show each one's
row in the raw, merged, and regression data.
```

The Test-First Cycle

Repeat for every component: merge, variables, model, table.

1. **You state the expectations:** files, keys, row counts, edge cases
2. **Claude Code writes the tests first.** You read them before any analysis code exists
3. **It implements the component** and runs the tests
4. **Failures are investigated, not patched:** fix the code, not the expected value
5. **Re-run the earlier tests** before moving to the next component

Why tests first

Tests written after the code tend to encode what the code does. Tests written first encode what *you* expect.

Test the Tests

A test that cannot fail is not a test. Inject a known error and confirm the check catches it.

Fault injection (always on a copy, never the raw data):

- Duplicate one survey respondent's row in a copy of the demographics file: the row-count and key tests should fail
- Change one treatment label in a copy: the spot check should fail

Planted answer: simulate outcomes with a known treatment effect. The estimator should recover it within a stated tolerance.

Example prompt:

```
Copy respondent_demographics.csv with one survey
respondent's row duplicated. Run the merge tests on the
copy and show me which tests fail.
```

Passing Is Not the Goal

Claude Code must never make a test pass by:

- Editing expected values to match the output
- Deleting or loosening assertions
- Suppressing warnings
- Changing the specification: join type, reference category, sample, weights

Not every check is pass/fail:

- **Hard failure** (duplicate keys): stop
- **Warning** (missingness jumps from 4% to 11%): flag it for you
- **Judgment** (is identification credible?): yours

A failing test is information

Claude Code should report it and ask. It should not make it disappear.

The Requirements-to-Tests Matrix

Requirement	Test	Passing condition
Merge keeps every respondent	Row-count and key test	2,044 rows; `respondent_id` unique
Outcome built as defined	Variable-construction test	`supports_intervention` is 1 exactly for "some" or "big"
Control is the reference arm	Specification test	The omitted category is `cntrl`
Survey weights are used	Independent recomputation	Logistic of the base-model intercept equals the weighted control share from the raw CSV
N is explained for every model	Sample accounting	Rows in − rows dropped (by reason) = N used
Every specification is reported	Output manifest	All specifications are in the table, each with its N

Write the matrix before the code. A requirement with no test is unverified.

The Audit Report: Evidence, Not "All Tests Passed"

```
## Audit report
Tests run: commands; passed / failed / skipped
Requirements: each requirement -> its test
Not tested: anything without a test, and why
Numerical checks: our value vs. independent value
Needs your call: choices Claude proposed
Not covered: identification, construct validity
```

What the report is for

Not for Claude Code to declare the research correct. It documents what was exercised, against which expectation, and what remains uncertain.

Make It a Rule in CLAUDE.md (1/2)

Do not rely on remembering to ask. Your protocol, part 1:

```
## Verification protocol (required)
### Before implementing a step
- State the expected inputs, keys, and row counts.
- Write the verification tests first and show them to me.
- Ask me before settling any research-design choice
  (join type, reference category, sample, weights).
  Mark any choice you proposed yourself.

### Tests must be independent of the code they check
- Take expected values from the raw files, a different
  method, or values I give you. Never paste the
  pipeline's own output in as the expected value.
- Every step gets a row-count reconciliation (rows in -
  rows dropped, by reason = rows out), data-integrity
  checks, and a trace of at least 3 records from the
  raw file to the final output.
```

Make It a Rule in CLAUDE.md (2/2)

Part 2 of the protocol:

```
### Never make a failing test pass by
- Editing expected values, deleting or loosening
  assertions, suppressing warnings, or changing the
  specification. Report the failure instead.

### Before saying a step is done
- Run the tests and show me the output.
- Report what passed, failed, and was skipped, and what
  remains unverified or needs my judgment.
- Passing tests do not show the research design is
  valid. Say what the tests do not cover.
- If you cannot write an independent test, say so.
```

Full text: `docs/verification_protocol.md`. CLAUDE.md is an instruction, not enforcement: read the tests too.

Two More Safeguards

Replicate before you extend

1. Give Claude Code the original code or description
2. Ask it to replicate a key table
3. Compare with the published result
4. Only then begin your extension

If you cannot reproduce the baseline, nothing built on it is interpretable.

Use a second agent as a critic

1. Agent 1 writes the code
2. Open a new conversation (Agent 2) and ask: "Read the analysis script and identify any bugs, silent data loss, or decisions that could change the results."

Agent 2 has fresh context and no attachment to the code.

Treat your agents like co-authors who check each other's work, not a single assistant you trust unconditionally.

Step 4: Save or Re-instruct

If the output passes verification: keep it. Claude Code saves its changes to your files after each step, so the results are already on disk.

If something is wrong: identify the specific error and re-instruct. Do not ask Claude Code to "fix it" — tell it *what* to fix and *how*.

In Session 3 we will learn **git** — a tool that lets you snapshot your project before each change and revert if anything goes wrong. For now, know that the save step exists and that there is a better way to manage it coming soon.

The full cycle

Describe → Review plan → Generate & validate → Save or re-instruct

Repeat for each task in your analysis. One cycle per task, not one cycle for the whole project.

What to Delegate, What to Verify, What to Keep

Delegate freely

Boilerplate & docs
File management†
Reformatting code

Delegate + verify

Data loading & cleaning
Data merges
Variable construction
Summary statistics
Robustness variants
Visualization
LaTeX tables

Do not delegate

Choosing specifications
Interpreting results
Identification strategy
Sample selection
Causal claims

† Be careful with file management — Claude Code can delete files. Always review before approving.

The test: *If a referee questioned this choice, who would need to answer — you or the RA?*

Today's exercise focuses on the middle column: tasks where Claude Code is productive but where you must verify every output.

Today's Data: The Swiss Franc Shock

Background: In January 2015, the Swiss National Bank removed its currency peg. The Swiss franc surged against the Polish zloty. Polish households with CHF-denominated mortgages saw their payments spike overnight.

The survey: 2,044 Polish adults, interviewed October 2015. Respondents were randomly assigned to receive different information about the crisis before being asked whether the government should intervene to help affected borrowers.

Your files:

- `swiss_franc_survey.csv` — 2,044 respondents, 11 variables
- `respondent_demographics.csv` — income and ideology data (more rows than the survey)
- `codebook.md` — variable descriptions

Source: Ahlquist, Copelovitch, and Walter, "The Political Consequences of External Economic Shocks: Evidence from Poland," *American Journal of Political Science* (2020).

The Research Question

Does providing information about the Swiss franc crisis increase support for government intervention to help affected borrowers?

Treatment (randomized):

- Control (no info)
- Factual information
- Historical context
- Hungary's policy response

Outcome:

- Supports government intervention (binary)

Moderator:

- FX loan exposure status

The methods are straightforward. The skill is staying in control of the workflow.

Warm-Up: Install the Verification Protocol (10 min)

1. **Add the protocol to your project.** Copy `docs/verification_protocol.md` into a file named `CLAUDE.md` in your `session_2` folder. If you already have one, append it.
2. **Check that Claude Code sees it.** Start Claude Code in the folder and ask: *"What verification rules apply in this project?"*
3. **Write a requirements-to-tests matrix by hand,** without Claude Code. For the merge you are about to do: three rows of requirement, test, and passing condition.

You write the expectations. Claude Code writes the tests.

Exercise Part 1: Merge and Explore (30 min)

Use plan mode for every step. Review every plan before approving.

1. **Merge** the demographics data into the survey.
   - You decide the join type. Do not let Claude Code choose.
   - Tests first: Claude Code writes the tests from your matrix, and you read them before it writes the merge
2. **Explore** the merged data: summary statistics, treatment group sizes, outcome distribution. First write down what you expect to see.
3. **Verify** the merge.
   - Run the tests. Trace three respondents: typical, missing outcome, absent from the demographics file
   - Test the tests: run them on a *copy* of the demographics file with one survey respondent's row duplicated. They should fail.

Part 1 Debrief: What Did You Verify?

What did you check?

Part 1 Debrief: What Did You Verify?

What did you check?

You should have verified:

- Row count: exactly 2,044 rows (no rows gained or lost)
- Join type: only a left join gives the correct result
- No duplicate respondent IDs in the output
- Treatment groups: 480 / 549 / 520 / 495
- Proportion supporting intervention: $\sim$0.479 (unweighted)
- FX exposure: 1,918 / 69 / 55
- Test the tests: the duplicated ID made the row-count and key tests fail

Did your numbers match? Did Claude Code choose the right join type?

Exercise Part 2: Specify and Estimate (30 min)

4. **Base model:** Logistic regression of `supports_intervention` on `treatment`.
   - Use survey weights. What should the reference category be? Tell Claude Code explicitly.
   - Independent check: the logistic of the intercept should equal the weighted control-group share, computed straight from the raw CSV
5. **Add FX exposure.** Did the sample size change? Why or why not?
6. **Add demographic controls:** age, female, education, urban/rural, income, left-right.
   - Before running: predict how many observations you will have. Then check.
7. **Add an interaction** between treatment and FX exposure.

For every model, reconcile N: rows in − rows dropped (by reason) = N used.

Part 2 Debrief: What Happened to Your Sample?

How many observations did you have in each model?

Part 2 Debrief: What Happened to Your Sample?

How many observations did you have in each model?

The Missing Data Trap

Adding control variables can silently change your sample.

Specification	N	What happened
Treatment only	2,036	Full sample (8 missing outcome)
+ FX exposure	2,035	2 missing FX status
+ age, female, education, urban/rural	2,035	No additional missingness
+ income quintile	∼1,473	565 missing income
+ left-right scale	∼1,499	542 missing left-right
+ both	∼1,151	44% of sample dropped

Did Claude Code mention this when it added the controls?

Exercise Part 3: Robustness and Output (30 min)

8. **Generate robustness checks** from your base model:
   - With and without survey weights
   - Different control sets (track how N changes each time!)
   - Collapse three treatment arms into "any information" vs. control
   - Restrict to complete cases so N is constant across specs
9. **Format as LaTeX.** Produce a publication-style regression table with all specifications side by side, and compile to PDF.
   - Output manifest test: every specification appears in the table, each with its N
10. **Audit report.** Ask Claude Code for `output/audit_report.md`, in the format from the audit-report slide.
11. **Commit.** Save all scripts, tests, output, and the LaTeX table.

The Robustness Workflow

You specify the core analysis.
Claude Code generates the variations.
You verify each one.

Claude Code never chooses what robustness checks to run.

For each specification, check:

- Does it use the right sample? (What is N?)
- Does the code match what you asked for?
- Are the coefficients in a plausible range?
- Do the signs on control variables make sense?

LaTeX Tables: Bringing Results Into Your Paper

Until now, your results are printed to the console or saved as CSVs.
In your actual workflow, they need to end up in a paper.

Why this matters:

- Claude Code can write LaTeX tables directly from regression output
- The table lives in your project directory, under version control
- When you update the analysis, Claude Code updates the table
- No manual reformatting, no copy-paste errors

Today: generate 5 tables instead of one table. In Session 3, we bring the full paper into VS Code.

The bigger idea

The more of your research that lives inside the project directory, the more Claude Code can operate on. Code, data, output, tables — and eventually the paper itself.

What to Watch For During the Exercise

This exercise is designed to surface moments where Claude Code makes choices for you.

- **The merge:** Did Claude Code pick the right join type? Did it mention the extra rows?
- **Reference categories:** Did it set treatment to "control" or pick alphabetically?
- **Missing data:** When you added controls, how many observations did you lose? Did Claude Code mention this?
- **Survey weights:** Did it use them unless you told it to?
- **Income = 0:** "No income" is coded as 0. How did Claude Code treat it?
- **Tests:** Did it copy the pipeline's own output in as an expected value, or edit one to get a pass?

These are the moments where your judgment matters.

The workflow is: notice the decision, direct Claude Code explicitly, then verify the output.

Debrief

Discussion questions:

1. Did your merge lose any observations? How did you handle weekends?
2. How many observations did you lose when you added controls? Did Claude Code tell you?
3. How many of your robustness specifications were correct on the first try?
4. What did you catch in the plan that you would have missed in the output?
5. Did any test fail? Did Claude Code fix the code, or change the test?

Key takeaway

The tests, the assertions, and the plan review are not extra work — they are the workflow. The 30 seconds it takes to read a plan or check a sample size can save hours of debugging downstream.

Next Session

Session 3: Code Auditing, Git, and LaTeX in VS Code

- Claude Code reads and critiques *your* code instead of writing it
- Git as your safety net: commit $\rightarrow$ instruct $\rightarrow$ diff $\rightarrow$ revert
- Bringing your paper into the project: LaTeX in VS Code

Before next session:

- Try the core workflow on something from your own research
- Note where Claude Code made a decision you did not ask for
- Confirm the LaTeX Workshop extension and a TeX distribution are installed (setup guide, Steps 1 and 7)